

LOG430 – Architecture Logicielle (Été 2025)

Laboratoire 4 - Auto-évaluation détaillée

Nom : _____ Code permanent : _____

Informations générales

- URL du dépôt GitHub/GitLab :
- Outil de test de charge utilisé (JMeter, k6, Artillery...) :
- Outil(s) de monitoring utilisé(s) (Prometheus, Grafana, Spring Actuator...) :
- Outil de load balancing (NGINX, HAProxy, etc.) :
- Mécanisme de cache implémenté (local, Redis, etc.) :

1. Évaluation technique par composant

1.1 Instrumentation et observabilité initiale

Critère	Oui	Non	Commentaire / Justification
Des logs structurés ont été intégrés à l'application			
Un endpoint de métriques est exposé (Prometheus, Actuator, etc.)			
Les métriques de base sont correctement collectées (requêtes, erreurs, temps de réponse)			
Les logs permettent de tracer les requêtes de bout en bout			

1.2 Monitoring et visualisation (Grafana)

Critère	Oui	Non	Commentaire / Justification
Un serveur de visualisation des métriques a-t-il été déployé et connecté à une source de collecte de données pour faciliter le suivi du système ?			
Un ou plusieurs dashboards ont été configurés à l'aide de templates existants ou personnalisés			

Les 4 Golden Signals ¹ sont représentés visuellement			
Les résultats des tests sont documentés (captures, exports de données, analyses)			

1.3 Test de charge initial (baseline)

Critère	Oui	Non	Commentaire / Justification
Des scénarios de test pertinents ont été définis (lecture/écriture, simultanéité, etc.)			
Des données réalistes ou représentatives ont été injectées			
Les métriques de performances ont été collectées en condition de charge			
Une analyse de la saturation ou des goulots d'étranglement a été menée			

1.4 Load Balancing

Critère	Oui	Non	Commentaire / Justification
Un répartiteur de charge a été mis en place (NGINX, HAProxy, etc.)			
Plusieurs instances de service ont été déployées derrière le répartiteur			
Les tests de charge ont été répétés avec load balancing activé			
Les métriques avant/après ont été comparées rigoureusement			
Un comportement de tolérance aux pannes a été testé (arrêt d'une instance)			

1.5 Caching applicatif

Critère	Oui	Non	Commentaire / Justification
Les endpoints critiques ont été identifiés (latence élevée ou forte fréquence)			
Un mécanisme de cache a été ajouté (ex : @Cacheable, Redis...)			

1. <https://sre.google/sre-book/monitoring-distributed-systems/>

Des règles d'expiration, d'invalidation ou de cohérence ont été définies			
L'impact du cache a été mesuré en termes de latence et de charge serveur			

2. Réflexion personnelle approfondie

1. **Instrumentation** : Quelle stratégie avez-vous adoptée pour instrumenter votre système ? Quels types de métriques vous ont semblé les plus révélateurs ?
2. **Monitoring** : Dans quelle mesure l'utilisation de visualisations (par exemple via Grafana) peut-elle aider à identifier des faiblesses ou des comportements inattendus ?
3. **Load Balancing** : Quelles sont les limites d'une simple répartition de charge dans le contexte de votre architecture ? Quelles optimisations complémentaires envisageriez-vous ?
4. **Caching** : Quels ont été les bénéfices mesurés du cache ? Quels risques potentiels avez-vous identifiés (cohérence, obsolescence, etc.) ?
5. **Approche séquentielle** : En comparant chaque étape (baseline, balancing, cache), laquelle a eu l'effet le plus significatif sur les performances ? Pourquoi ?
6. **Professionnalisation** : Quelles compétences ou outils appris dans ce laboratoire pensez-vous réutiliser dans un projet réel en entreprise ?